

Today's Goals

1. Understanding information flow



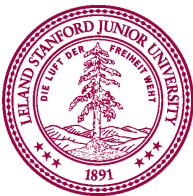
Bad Times With functions

NOTE: This program is buggy!!

```
def add_five(x):  
    x += 5
```

```
def main():  
    x = 3  
    add_five(x)  
    print("x = " + str(x))
```

Let's "trace"
this program

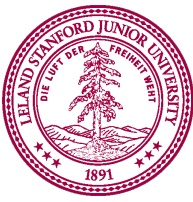


Bad Times With functions

NOTE: This program is buggy!!

```
def add_five(x):  
    x += 5
```

```
def main():  
    x = 3  
    add_five(x)  
    print("x = " + str(x))
```

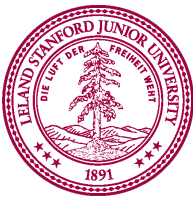


Good Times With functions

NOTE: This program is **feeling just fine...**

```
def add_five(x):  
    x += 5  
    return x
```

```
def main():  
    x = 3  
    x = add_five(x)  
    print("x = " + str(x))
```



Good Times With functions

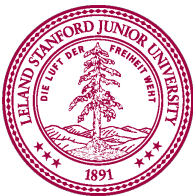
NOTE: This program is feeling just fine...

```
def add_five(x):  
    x += 5  
    return x
```

```
def main():  
    x = 3  
    x = add_five(x)  
    print("x = " + str(x))
```



When we want to “reassign” x inside a helper function, employ the `x = change(x)` pattern!

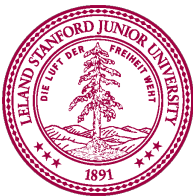


Parameter passing mechanism



When a parameter is passed during a function call, a **new variable** is created for the lifetime of the function call.

That new variable may or may not have the **same name** as the value that was passed in!



No inherent connection between these two

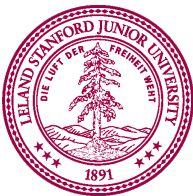
NOTE: This program is buggy!!

```
def add_five(x):  
    x += 5
```

These are two
separate variables.
They are not linked!

```
def main():  
    x = 3  
    add_five(x)  
    print("x = " + str(x))
```

Only relationship:
value of main's x is
used when creating
add_five's x





Later on in class... we will see cases where changes to variables in helper functions seem to persist! It will be great. We will let you know when we get there and exactly when that happens!



Careful!

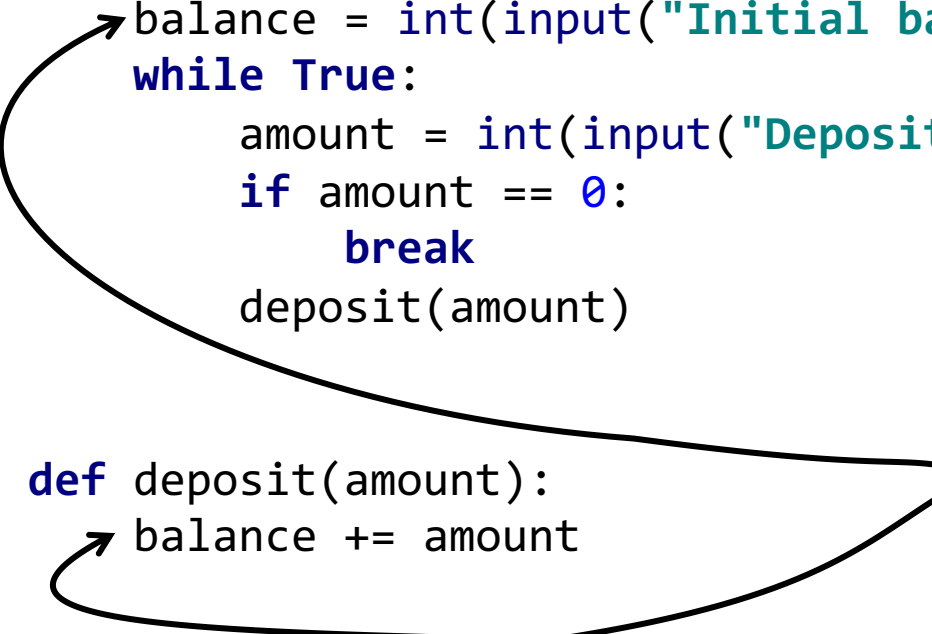
Reviewing Parameters and
information flow

Referring to Variables in Functions

```
def main():  
    balance = int(input("Initial balance: "))  
    while True:  
        amount = int(input("Deposit (0 to quit): "))  
        if amount == 0:  
            break  
        deposit(amount)
```

```
def deposit(amount):  
    balance += amount
```

Different variables with the same name!



Use return to pass back information



Don't want using your toaster
to impact your refrigerator!



```
def main():  
    balance = int(input("Initial balance: "))  
    while True:  
        amount = int(input("Deposit (0 to quit): "))  
        if amount == 0:  
            break  
        balance = deposit(balance, amount)
```

```
def deposit(balance, amount):  
    balance += amount  
    return balance
```

Encapsulation Principle



Don't want using your toaster
to impact your refrigerator!



```
def main():  
    balance = int(input("Initial balance: "))  
    while True:  
        amount = int(input("Deposit (0 to quit): "))  
        if amount == 0:  
            break  
        balance = deposit(balance, amount)
```

```
def deposit(balance, amount):  
    balance += amount  
    return balance
```

Encapsulation Principle: Data
used by a function should be a
constant, a parameter or a (local)
variable encapsulated in function